MODULE 19 · DOMAIN 3 - DEPLOYMENT, PROVISIONING, AND AUTOMATION

CloudFormation Operations

Stack states, change sets, drift, nested stacks, StackSets – and getting out of a failed rollback.

DOMAIN 3 of 5 EXAM WEIGHT 22% MODULE 19 of 35

admin@sthithapragna.com

A stack is a **state machine**. The state you are in determines what operations are legal, and one state permits only deletion.

The states that matter

STATE	WHAT YOU CAN DO FROM HERE
CREATE_COMPLETE / UPDATE_COMPLETE	Everything. Update, delete, create change sets
ROLLBACK_COMPLETE	DELETE ONLY. The stack cannot be updated
UPDATE_ROLLBACK_COMPLETE	Normal operations resume. This one is fine
UPDATE_ROLLBACK_FAILED	Continue update rollback, optionally skipping resources
DELETE_FAILED	Retry the delete, optionally retaining the resources that blocked it
REVIEW_IN_PROGRESS	A change set exists but has never been executed

ON THE EXAM

ROLLBACK_COMPLETE and UPDATE_ROLLBACK_COMPLETE sound almost identical and are completely different. One is a dead end, the other is recovery.

ROLLBACK_COMPLETE: the dead end

- It means the **initial creation** failed and the rollback succeeded. The stack exists as an empty shell.
- **You cannot update it.** The only permitted operation is delete.
- Fix the template, **delete the stack, and create it again**. There is no repair path.
- It only ever applies to a **first creation**. A stack that succeeded once will never reach this state.

> ON THE EXAM
>
> "The stack is in ROLLBACK_COMPLETE and my update fails" is not a permissions problem or a template problem. That state permits nothing but deletion.

When the rollback itself fails

- **UPDATE_ROLLBACK_FAILED** means an update failed, and putting things back also failed. Usually an external change or a dependency.
- **ContinueUpdateRollback** retries. If a specific resource keeps blocking it, use **ResourcesToSkip**.
- Skipping leaves that resource **out of sync with the template**, so fix it manually afterwards and confirm with drift detection.
- Common cause: somebody deleted or modified a resource outside CloudFormation while the stack was mid-update.

> ON THE EXAM
>
> Continue update rollback is the only way out. Deleting the stack from this state usually fails for the same reason the rollback did.

Failing without rolling back

- **Disable rollback** on a create or update keeps successfully provisioned resources in place when something fails.
- That lets you **inspect the half-built stack** and read the actual error on the resource, rather than watching it disappear.
- After fixing the cause, you can **retry the operation** from where it stopped rather than starting again.
- The stack sits in a failed state meanwhile, so this is a debugging tool rather than a production default.

> ON THE EXAM
>
> The classic frustration is a resource failing, rolling back, and being deleted before anyone could look at it. Disabling rollback is the fix for that.

Rollback triggers

- **Rollback triggers** name CloudWatch alarms that CloudFormation monitors during a create or update.
- If any named alarm enters ALARM, CloudFormation **rolls back the whole operation automatically**.
- Monitoring continues for a period **after** the operation completes, up to 180 minutes, so a delayed failure still triggers a rollback.
- Point them at real signals: application error rate, latency, or a business metric.

> ON THE EXAM
>
> This is how a deployment becomes self-reversing. The stack succeeding is not the same as the application still working.

Change sets

- A **change set** is a preview: what will be added, modified or removed, and crucially whether each change causes **replacement**.
- The **Replacement** column reads True, False or **Conditional**. Conditional means it depends on values CloudFormation cannot evaluate yet.
- Creating a change set on a new stack puts it in **REVIEW_IN_PROGRESS** until you execute it.
- Since 2025, CloudFormation runs **pre-deployment validation** at change set creation, catching property syntax errors and name conflicts early.

> ON THE EXAM
>
> Read the Replacement column before every production update. It is the difference between changing a database and creating a new empty one.

New: drift-aware change sets

- Announced **18 November 2025**. A standard change set compares the new template with the last deployed one, and ignores drift.
- A drift-aware change set does a **three-way diff**: new template, last deployed template, and **actual infrastructure state**.
- On execution it **reverts drift** to the template values, and **recreates resources deleted outside CloudFormation**.
- Create it with change set type **Drift-aware**, or **--deployment-mode REVERT_DRIFT**.

> ON THE EXAM
>
> Before this, reverting drift meant detecting it, then working out by hand what to change. Now the existing template is the remediation.

Drift detection

- Compares the **actual configuration** of stack resources against the template. Per resource: IN_SYNC, MODIFIED, DELETED or NOT_CHECKED.
- **Not every resource type supports it**, and unsupported ones report NOT_CHECKED rather than failing.
- It is **on demand**, not continuous. Run it on a schedule, or trigger it from EventBridge.
- StackSets has its own drift detection across every stack instance, which is how you find one account that was edited by hand.

> ON THE EXAM
>
> Drift detection tells you something changed. Drift-aware change sets are how you put it back. Detection and remediation are now separate features.

Nested stacks or cross-stack exports?

NESTED STACKS

One stack, composed

- A parent template referencing child templates
- Deployed, updated and deleted as one unit
- Values passed as parameters, returned as outputs
- For reusable components in one application

CROSS-STACK EXPORTS

Separate, loosely joined

- Independent stacks with their own lifecycles
- Export and Fn::ImportValue between them
- For different teams owning different layers
- Rigid: an export in use cannot change

THE TIEBREAKER

Ask who owns the lifecycle. One team deploying one application is nested stacks. A network team and an application team is cross-stack.

Nested stack mechanics

- A child is an **AWS::CloudFormation::Stack** resource whose TemplateURL points at S3.
- **Update from the root stack**, never the child directly. Updating a child directly makes the parent drift from reality.
- The **root stack** is the top of the tree; a **parent** is the immediate stack above a given child.
- Nested stacks each count toward the resource limit separately, which is how large architectures get past the 500-resource ceiling.

> ON THE EXAM
>
> Deleting the root deletes every nested stack. That is the point of nesting, and it is also why you never nest a database you must keep.

Bringing existing resources under management

- **Resource import** brings an existing resource into a stack without recreating it, matched by its physical identifier.
- **The IaC generator** scans an account and produces a template from resources that already exist.
- It works within **one account and one Region**, and generates **literal values** rather than parameters.
- Expect a refactoring pass afterwards to introduce parameters, mappings and conditions. The generated template is a starting point.

> ON THE EXAM
>
> "We built it in the console and now we need it in code" is the IaC generator, followed by resource import. Neither recreates anything.

Stack refactoring

- **Move resources between stacks**, or rename their logical IDs, **preserving state and data**.
- Without it, moving a resource meant removing it from one template and adding it to another, which **deletes and recreates** it.
- Available through the CLI, and since 2025 through the **console and CDK** as well.
- The typical use: splitting one large stack that has outgrown itself, or extracting a shared component.

> ON THE EXAM
>
> Renaming a logical ID used to be a destructive operation. That surprised a lot of people, and refactoring is the supported way to avoid it.

StackSets

- One template, deployed as **stack instances** across many accounts and Regions from a single administrator account.
- The **StackSet** is the definition. A **stack instance** is one deployment of it into one account and Region.
- Updating the StackSet updates every instance, which is what stops ninety accounts drifting apart.
- A stack instance can fail independently. Failure tolerance decides whether that stops the whole operation.

> ON THE EXAM
>
> StackSets solves consistency across accounts. Nested stacks and exports solve structure within one account. They answer different questions.

Self-managed or service-managed permissions?

SELF-MANAGED

You create the roles

- An administration role in the admin account
- An execution role in every target account
- Works with accounts outside an organisation
- You add each target account explicitly

SERVICE-MANAGED

Organizations handles it

- Target organisational units, not account IDs
- AWS creates and manages the roles
- New accounts joining an OU are covered automatically
- Needs Organizations, all features enabled

THE TIEBREAKER

"New accounts must be covered automatically" is service-managed. That automatic deployment is the feature self-managed cannot offer.

StackSets operation preferences

- **Maximum concurrent accounts**: how many accounts are updated at once, as a number or a percentage.
- **Failure tolerance**: how many accounts may fail before the whole operation stops. Set it low for production.
- **Region concurrency**: sequential or parallel, plus a **Region order** so you can deploy to a test Region first.
- Failure tolerance of 0 with concurrency of 1 is the safest and slowest. Across 200 accounts, that is a very long afternoon.

> ON THE EXAM
>
> These are the StackSets equivalent of Run Command's rate control and instance refresh's checkpoints. Same idea, different service.

Operating StackSets

- **Delegated administration** lets a member account run StackSets, so the management account is not used for daily work.
- **Drift detection for StackSets** checks every stack instance, which is how you find the one account somebody edited by hand.
- **Parameter overrides** let one account differ, for example a different CIDR range, without a separate StackSet.
- Overrides are also how StackSets quietly become inconsistent. Use them sparingly and deliberately.

> ON THE EXAM
>
> Organization backup policies from module 16 and StackSets solve the same class of problem: push configuration down so accounts cannot opt out.

Git sync, and getting unstuck

- **CloudFormation Git sync** watches a repository and deploys a change set whenever a template or deployment file changes.
- Since September 2024 it posts **change set details as pull request comments**, so the review shows what will actually change.
- **Termination protection** blocks stack deletion until it is explicitly turned off. Enable it on production stacks.
- For a stack that will not delete, **force delete** options can retain the blocking resources and remove the stack.

> ON THE EXAM
>
> A pull request showing the change set is the cheapest safety control in this module. The reviewer sees replacement warnings before merging.

> A developer creates a new stack. It fails because an S3 bucket name was already taken, and CloudFormation rolls back. The stack now shows ROLLBACK_COMPLETE.
>
> The developer corrects the bucket name in the template and attempts a stack update, which fails immediately.

What should the developer do?

A Use ContinueUpdateRollback to clear the failed state

B Delete the stack and create it again with the corrected template

C Create a change set against the existing stack and execute it

D Enable disable-rollback and retry the update

◆ PAUSE AND DECIDE

What should the developer do?

A ContinueUpdateRollback

Applies to UPDATE_ROLLBACK_FAILED, a different state. There is no update rollback in progress to continue here.

B Delete and recreate

ROLLBACK_COMPLETE permits deletion only. There is no repair path, and the stack contains nothing worth preserving.

C Change set and execute

A change set is still an update, and updates are not permitted from this state. It fails for the same reason.

D Disable rollback and retry

Useful on the next attempt to keep resources for debugging. It does not make an update legal from ROLLBACK_COMPLETE.

THE SIGNAL

The similar names are the whole trap. ROLLBACK_COMPLETE is a first-creation dead end; UPDATE_ROLLBACK_COMPLETE and UPDATE_ROLLBACK_FAILED are different states entirely.

During an incident, an engineer manually increased an Auto Scaling group's maximum size and changed a security group rule, both outside CloudFormation. The stack's template still holds the original values.

The team wants to bring the resources back in line with the template, and wants to see exactly what will change before doing it.

What should the team use?

A. A drift-aware change set, reviewing the three-way diff before executing

B. Drift detection, then manually edit each resource back to its template value

C. A standard change set with no template changes

D. Delete the stack and redeploy it from the template

◆ PAUSE AND DECIDE

What should the team use?

A Drift-aware change set

The three-way diff shows template, last deployment and actual state, then execution reverts the drift. It answers both halves of the requirement.

B Detect, then edit by hand

What teams did before November 2025. It works, it is manual, and it is easy to miss a property.

C Standard change set

The sharpest distractor. A standard change set compares the new template with the last deployed one and does not consider drift at all, so it reports no changes.

D Delete and redeploy

Destroys and recreates everything to correct two properties. Maximum disruption for a small correction.

THE SIGNAL

Option C fails for a specific reason worth remembering: a standard change set is blind to drift. That blindness is exactly what the new type removes.

> An organisation with 140 accounts must ensure every account has a standard set of IAM roles and a Config recorder. New accounts are created roughly weekly and must be covered without anyone remembering. The organisation uses AWS Organizations with all features enabled.
>
> Deployment must stop if more than two accounts fail.

What should the organisation implement?

A. A service-managed StackSet targeting the organisation's OUs, with automatic deployment enabled and failure tolerance set to 2

B. A self-managed StackSet listing all 140 account IDs

C. A CloudFormation template deployed by a pipeline into each account

D. Nested stacks, with one child stack per account

◆ PAUSE AND DECIDE

What should the organisation implement?

A Service-managed StackSet on OUs

Automatic deployment covers accounts created later with no action, and failure tolerance of 2 implements the stated stopping condition.

B Self-managed with 140 account IDs

Works today and requires somebody to add each new account every week. That is precisely the manual step the requirement forbids.

C A pipeline per account

Rebuilds StackSets by hand, with no failure tolerance mechanism and 140 deployments to keep aligned.

D Nested stacks

Nesting composes a template within one account. It is not a multi-account mechanism at all.

THE SIGNAL

"New accounts must be covered without anyone remembering" is automatic deployment, and only service-managed permissions offer it.

A single CloudFormation stack has grown to manage networking, databases and three applications. The team wants to split it into four stacks owned by different teams, without recreating the RDS databases or the VPC.

Removing resources from one template and adding them to another would delete and recreate them.

What should the team use?

A Stack refactoring, to move resources between stacks while preserving state

B Resource import into new stacks after deleting the original stack with resources retained

C Convert the stack to nested stacks with the same resources

D Export the resources and use Fn::ImportValue in the new stacks

◆ PAUSE AND DECIDE

What should the team use?

A Stack refactoring

Purpose-built for exactly this: move resources between stacks and rename logical IDs while preserving state and data.

B Retain, delete, then import

The pre-refactoring workaround. It works, and it is several careful manual steps with a window where nothing is managed.

C Convert to nested stacks

Restructures the template and keeps one lifecycle. The requirement is separate ownership by different teams, which nesting does not give.

D Export and ImportValue

Shares values between stacks. It does not move a resource from one stack's management into another's.

THE SIGNAL

Option B is the honest historical answer and the reason refactoring exists. Recognising the manual workaround and the supported feature is worth knowing.

> A stack update fails partway through and then enters UPDATE_ROLLBACK_FAILED. Investigation shows that an engineer deleted a security group by hand during the update, so CloudFormation cannot restore the previous state.
>
> The team needs the stack back into a usable state.

What should the team do?

A. Delete the stack and redeploy from the template

B. Call ContinueUpdateRollback, skipping the security group if it still blocks, then fix it manually and verify with drift detection

C. Create a change set to reapply the previous template

D. Enable termination protection and retry the update

◆ PAUSE AND DECIDE

What should the team do?

A. Delete and redeploy

Deleting from UPDATE_ROLLBACK_FAILED usually fails for the same reason the rollback did, and it destroys everything the stack manages.

B. ContinueUpdateRollback, then repair

The documented way out. Skipping leaves that resource out of step with the template, so the manual fix and drift check complete the job.

C. Change set on the previous template

Updates are not permitted while the stack is in UPDATE_ROLLBACK_FAILED. The rollback must be resolved first.

D. Termination protection

Prevents deletion. It has no effect on a failed rollback and does not make an update legal.

THE SIGNAL

Note the second half of the answer. Skipping a resource clears the state and leaves a lie in the template, so the repair is not finished at that point.

The five things to carry forward

- **ROLLBACK_COMPLETE permits deletion only.** UPDATE_ROLLBACK_FAILED needs ContinueUpdateRollback.
- **A standard change set is blind to drift.** Drift-aware change sets add actual state and revert it.
- **Nested stacks share one lifecycle; exports keep lifecycles separate.** Ask who owns what.
- **Service-managed StackSets deploy to new accounts automatically.** Self-managed cannot.
- **Stack refactoring moves resources without recreating them.** Renaming a logical ID used to be destructive.

> ON THE EXAM
>
> Next: beyond CloudFormation - CDK, Terraform, Git workflows, Service Catalog, Control Tower and RAM.